

TECHNISCHE UNIVERSITÄT DRESDEN
Faculty of Computer Science
Chair of Machine Learning for Computer Vision

PROJECT REPORT
Summer Semester 2026

Tracking-by-Clustering on a Synthetic Spacetime Dataset

A Greedy Multicut Approach and Its Empirical Accuracy

submitted by

Daniel MON
Matriculation no. 5356608

Muhammad TAHA
Matriculation no. 5311753

Examiner: Prof. Dr. Björn Andres
Consultant: David Stein

Dresden, 22 August 2026

Abstract

Multi-object tracking is conventionally split into a detection stage and an association stage, so a linking error committed in one frame cannot be repaired afterwards. Tracking-by-clustering removes that split: every detection from every frame becomes a vertex of a single graph, and a track is one cluster of that graph. This report builds such a system end to end on synthetic data and measures its accuracy under controlled degradation. Four spheres move on a two-dimensional flat torus under elastic collisions and two motion regimes, and are observed as noisy inlier detections embedded in uniform background clutter, yielding 7,800 points per run. Both the gate radius and the cost zero-crossing are estimated from the observed cloud itself via a median and median-absolute-deviation statistic, so no threshold is set by hand, and the resulting signed-cost instance is partitioned by greedy additive edge contraction in under 1.1 seconds. Accuracy is reported as the variation of information against the sealed ground truth, decomposed into split and merge error. All four objects are recovered, without their number being supplied to the tracker, up to an observation noise of $\sigma_{\text{obs}} = 0.20$, with merge error exactly zero below that level. Accuracy then degrades sharply, and a measured VI near 1.5, above the total-fusion ceiling $\ln 4 \approx 1.386$, establishes that the failure combines fragmentation with fusion rather than a simple collapse. An ablation shows the gate estimator depends on the very clutter it filters out. The pipeline is also exposed as an interactive application over the same code base. The no-split and no-join constraints are relaxed, and no bound on the optimality gap is reported.

Contents

Division of Work	3
1 Introduction	4
2 Methods	4
2.1 Data synthesis	4
2.2 Instance building: gating and edge costs	7
2.2.1 Why we cannot connect everything	7
2.2.2 Setting the gate from the data	8
2.2.3 From one threshold to two: gating versus cost	10
2.2.4 Calibrating the cost margin	11
2.3 Solving the instance	12
2.4 Constraint handling	14
2.5 Evaluation	14
2.5.1 Metric	14
2.5.2 What we vary	15
2.5.3 Procedure	16
2.6 The interactive application	16
3 Results	17
3.1 Observation noise and motion regime	17
3.2 Number of objects	18
3.3 Background clutter	19
3.4 Object speed	20
3.5 Inlier density	20
3.6 Pre-filter threshold	21
3.7 Failure anatomy	22
4 Discussion	22
4.1 What the results show	22
4.2 Why the two radii matter	22
4.3 Limitations	23
4.4 Outlook	23

Division of Work

This report is the work of two authors, and the division below is the same one used in the project presentation. Each author wrote the sections listed under their name and implemented the code underlying them. The abstract was assembled from both parts.

Part	Author	Sections
1	Daniel Mon	Section 1: the task, its constraints and the deliverables. Section 2.1: world simulation, motion model, elastic collisions, observation pipeline, background clutter. Section 2.2 including Sections 2.2.1–2.2.4: candidate edges, gate estimation from the observed cloud, the two radii, cost calibration.
2	Muhammad Taha	Section 2.3: greedy additive edge contraction, union–find, lazy-deletion heap, complexity. Section 2.4: transitivity by construction, the plain multicut relaxation. Section 2.5 and Section 3: evaluation protocol, variation of information, all measured results and ablations. Section 2.6: the interactive application. Section 4: interpretation, limitations and outlook.

On the code base, `synthesis.py`, `motion.py`, `collision.py`, `observation.py` and `instance.py` were written by Daniel Mon; `solver.py`, `app.py` and the evaluation harness by Muhammad Taha. `geometry.py` and `viz.py` were developed jointly.

1 Introduction

Multi-object tracking asks a simple question: given point detections over time, which detections belong to the same object, and how does each object move? The classical answer, *tracking-by-detection*, splits the work into two stages. First objects are detected in each frame, then those detections are linked across frames into trajectories [3]. The split is brittle. A linking error made in one frame cannot be repaired later on, and the two stages optimise separate objectives that need not agree with each other.

Tracking-by-clustering removes the split. All detections across all frames become the vertices of one graph, and a track is simply one cluster of that graph. Partitioning the graph solves detection and linking together, under a single objective, so within-frame evidence (which detections in a frame belong together) and between-frame evidence (which detections in consecutive frames continue an object) compete directly instead of being decided in sequence. No separate detection stage exists, because identity is never frame-local to begin with. Formally the partition is a minimum-cost multicut problem [3, 4], NP-hard in general but solvable in practice by efficient greedy heuristics [1].

This project builds such a system end to end and measures its accuracy under controlled conditions. Following the setup of Andres and Stein, we synthesise a dataset whose ground-truth trajectories are known but sealed from the tracker, turn the resulting anonymous point cloud into one multicut instance, solve it with a greedy solver of our own, and measure how well the recovered clusters match the hidden tracks as observation noise and background clutter vary. The ground truth is available but withheld during inference, which makes the experiment a genuine accuracy test rather than a reconstruction of known quantities.

The project is deliberately scoped to keep the measurement clean. The world is a two-dimensional flat torus $[0, L)^2$ with periodic spatial boundaries; time is linear, and edges connect only points within a frame and between consecutive frames, with no lifting to longer-range temporal edges. Objects are moving spheres under one of two motion regimes, an inertial random walk and its ballistic limit, and detections include both inlier points around each sphere and uniform background clutter. The tracker sees only detection positions and timestamps. Labels are revealed only at evaluation, where accuracy is reported as the variation of information (VI) between the predicted and true partitions, with the adjusted Rand index (ARI) as a secondary check, over the full grid of motion models, noise levels, and seeds. We use our own greedy solver rather than a general integer-program solver, and work in 2D space with linear time in place of a 3D spatial domain.

2 Methods

2.1 Data synthesis

We evaluate the tracker on synthetic data, where the ground truth is known but withheld. Generation has two stages. A simulation moves K spheres through a bounded world, producing the exact trajectory of each sphere. An observation stage then samples that world: each sphere yields a small cluster of noisy detections, and uniform background detections are added throughout. The tracker receives only the resulting point cloud and its timestamps, never the sphere identities, nor which detections are background. Those labels stay sealed until evaluation, which is what makes the experiment an accuracy test rather than a reconstruction of known quantities.

World geometry. The spheres move on a square domain of side L with periodic boundaries: a sphere leaving one edge re-enters on the opposite one, so the domain is a flat two-dimensional torus (a 2-torus; Figure 1a). The wrapping keeps the dynamics uniform, since no sphere ever reaches a boundary, but it changes how distance is measured. Two points near opposite edges are far apart across the interior yet close across the seam, so the correct separation is the shorter of the two. For each coordinate, the signed displacement from a to b is

$$\Delta_i = \left((b_i - a_i + \frac{L}{2}) \bmod L \right) - \frac{L}{2}, \quad i \in \{x, y\}. \quad (1)$$

The straight-across gap lies in $[0, L)$; whenever it exceeds half the domain, the wrapped path is shorter, and Equation (1) selects it automatically, returning a value in $[-L/2, L/2)$. It preserves sign, giving direction as well as magnitude, and that is what lets collisions act across the seams. Time does not wrap: the simulation runs forward, so the time axis is linear.

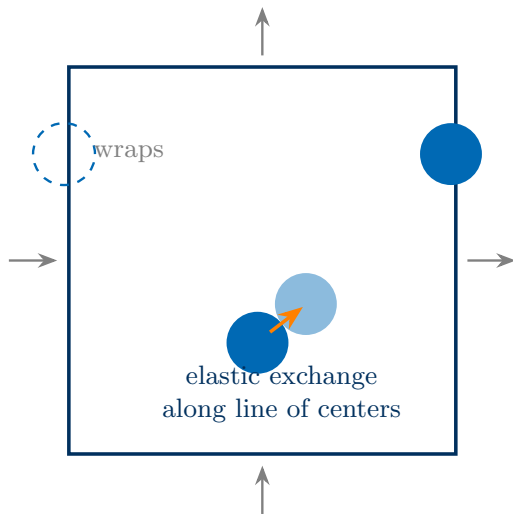
Motion model. Each sphere carries a velocity and advances once per frame by a semi-implicit Euler step, in which the velocity is updated first and then applied to the position:

$$v_k(t+1) = v_k(t) + \eta_k(t), \quad \eta_k(t) \sim \mathcal{N}(0, \sigma_{\text{mot}}^2 I_2), \quad (2)$$

$$c_k(t+1) = \text{wrap}(c_k(t) + v_k(t+1) \Delta t, L). \quad (3)$$

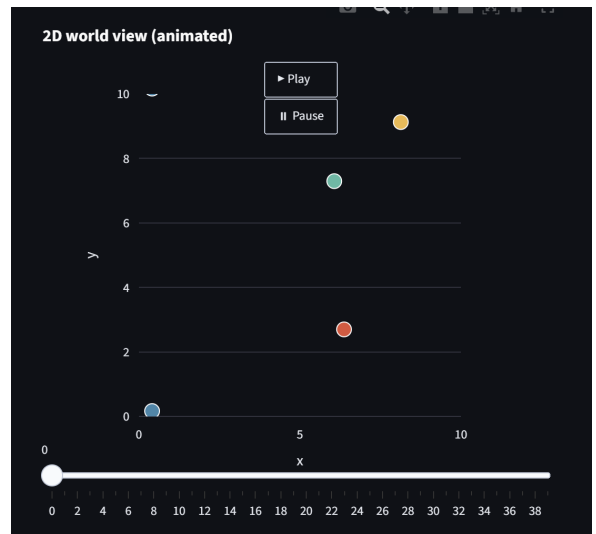
The noise scale σ_{mot} sets the regime. At $\sigma_{\text{mot}} = 0$ the velocity is constant, so spheres travel in straight lines and change direction only through collisions. We call this the *ballistic* regime. At $\sigma_{\text{mot}} > 0$ the velocity is perturbed each frame by a Gaussian increment, so the velocity itself follows a Gaussian random walk and the heading drifts, giving the *random-walk* regime. Evaluating both tests whether the tracker depends on smooth, predictable motion.

Collisions. Spheres of radius r undergo equal-mass elastic collisions. A pair is resolved only when it overlaps (torus distance below $2r$) and is approaching, the latter enforced by $\langle v_i - v_j, \hat{n} \rangle > 0$ along the line of centres $\hat{n}$; this second condition prevents a just-separated pair from colliding again while still overlapping. The velocity components along $\hat{n}$ are then exchanged. Since $\hat{n}$ derives from the displacement of Equation (1), collisions remain correct across the periodic seams.



world $[0, L)^2$, periodic in x and y

(a) The wrapping world.



(b) Spheres moving in the world.

Figure 1: The simulated world. (a) The square world wraps: left joins right and top joins bottom, forming a flat torus, so a sphere leaving one edge returns on the opposite one and collisions act across the seams. (b) A snapshot of the hidden ground-truth simulation: several spheres in motion, each later observed as a cloud of noisy detections.

Observation model. The simulation yields each sphere's true centre per frame, but the tracker observes only noisy detections. Each frame is sampled in two parts. Every sphere produces n_{in} *inlier* detections, each the true centre plus isotropic Gaussian noise (the centre plus an error term),

$$p = \text{wrap}(c_k(t) + \eta, L), \quad \eta \sim \mathcal{N}(0, \sigma_{\text{obs}}^2 I_2), \quad (4)$$

where the width σ_{obs} controls how tightly the detections concentrate around the centre. A further n_{bg} *background* (clutter) detections are drawn from a uniform distribution across the domain, belonging to no sphere and modelling false detections. Inliers carry their sphere’s identity; clutter carries the label -1 . Concatenating all frames gives the positions $P \in \mathbb{R}^{M \times 2}$, timestamps t , and sealed labels ℓ , with $M = T \cdot (K n_{\text{in}} + n_{\text{bg}})$. Only P and t are exposed to the tracker. The main experiments use $K = 4$ spheres over $T = 60$ frames, with $n_{\text{in}} = 20$ inliers and $n_{\text{bg}} = 50$ clutter points per frame, for $M = 7,800$ points in total; the interactive application (Section 2.6) defaults to a smaller world. Figure 3 shows one observed frame.

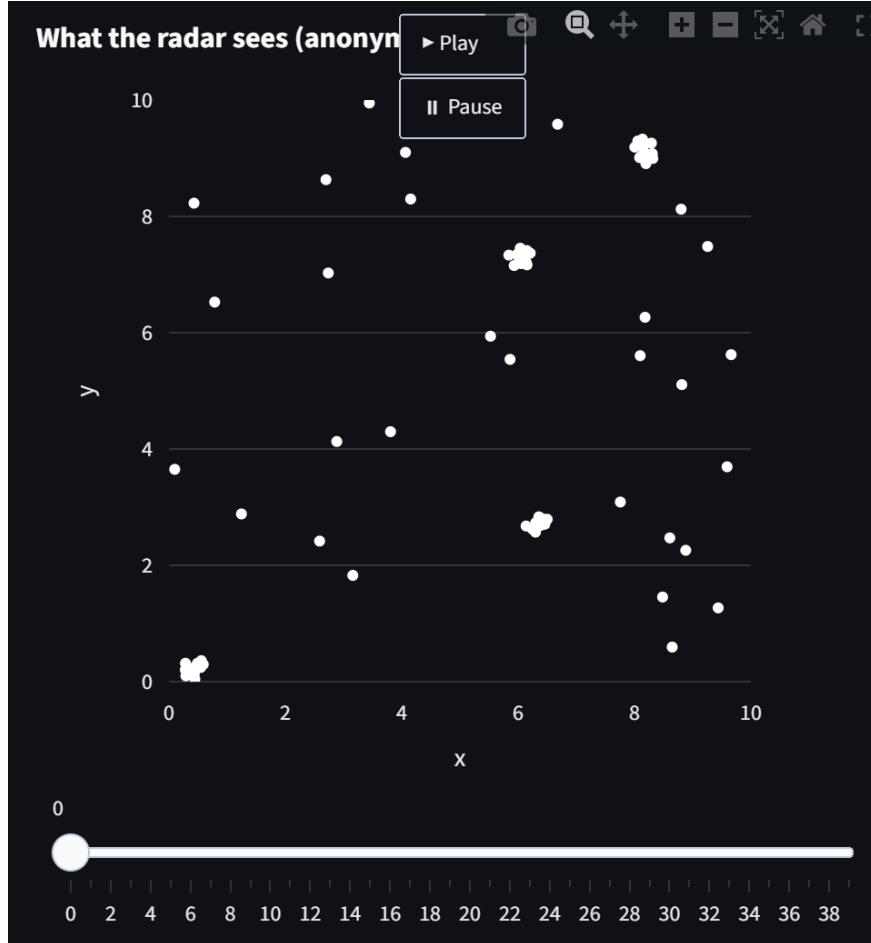


Figure 2: The observed point cloud as shown in the interactive application: the anonymous detections passed to the tracker, with inlier clusters and background clutter combined.

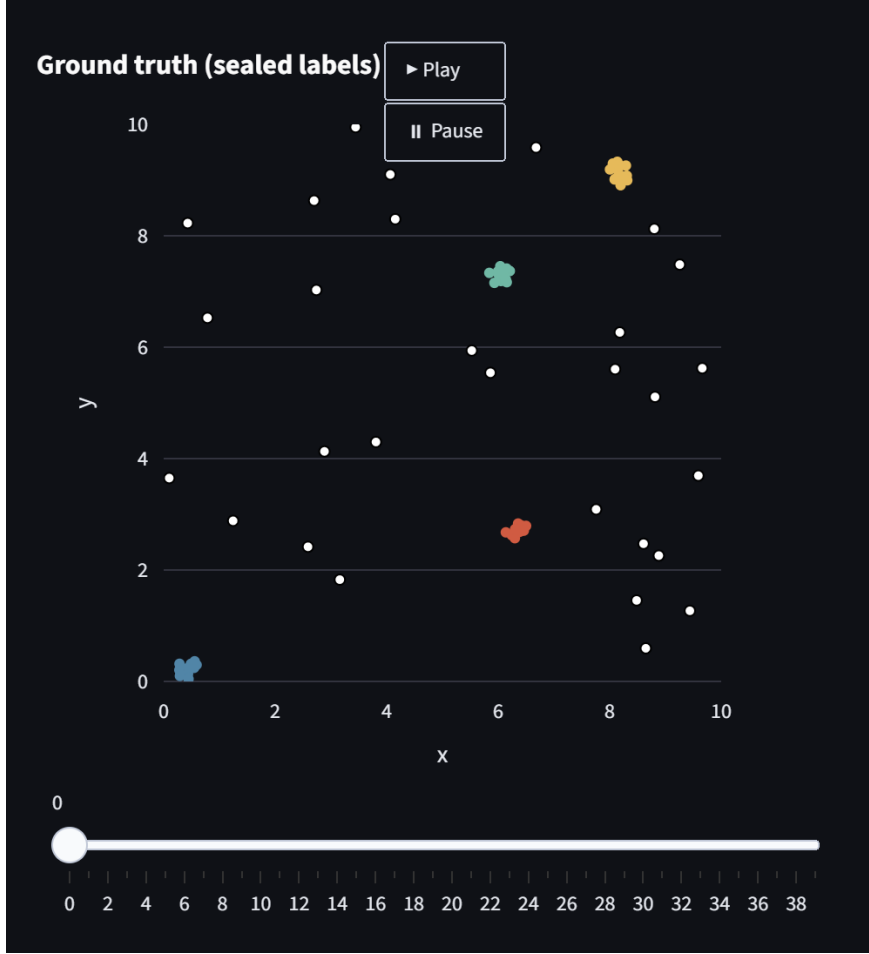


Figure 3: One observed frame as the tracker receives it: K dense inlier clusters (one colour per sphere here, for illustration) buried in uniform background clutter (grey). The tracker sees only the positions and timestamps, not the colours, which are the sealed ground truth.

2.2 Instance building: gating and edge costs

The tracker never detects objects and then links them. Instead it builds a single graph over *all* detections from all frames and partitions it: each resulting cluster is one track. This section builds that graph in three steps. First we decide which pairs of points are even worth connecting with an edge (gating). Then we assign each edge a cost, positive or negative, that says whether its two endpoints should end up in the same cluster. Finally we explain why gating and costing use two different thresholds rather than one, the design choice that makes the whole method work.

2.2.1 Why we cannot connect everything

In the graph, vertices are detections and an edge between two detections carries evidence about whether they belong to the same object. As stated in the introduction, partitioning this graph is a minimum-cost multicut problem. With a binary variable $y_e \in \{0, 1\}$ marking each edge as *cut* ($y_e = 1$, its endpoints in different clusters) or *joined* ($y_e = 0$), the problem is

$$\min_{y \in \{0,1\}^E} \sum_{e \in E} c_e y_e \quad \text{subject to} \quad y_e \leq \sum_{e' \in C \setminus \{e\}} y_{e'} \quad \forall \text{cycles } C, \forall e \in C. \quad (5)$$

The cycle inequalities enforce consistency: within any cycle, no single edge may be the only one cut, which is what guarantees the cut edges describe a valid partition. (The clique-partitioning formulation seen in the course is the special case of Equation (5) on a complete graph; gating, below, deliberately

keeps the graph sparse instead.) This problem is NP-hard in general, which is why we solve it with the greedy heuristic of Section 2.3 rather than exactly.

Everything in this section serves that optimisation, and it begins with a choice about which edges to even include. The simplest choice would be to place an edge between every pair of points. With M points that is $\binom{M}{2} = O(M^2)$ edges, and for our instances, where $M \approx 7,800$, that is roughly 3×10^7 edges. Building this complete graph is a bad idea for two separate reasons.

The first is cost. Every later step works through the edge set, both the computation of edge scores and the solver itself, so a graph that grows quadratically in the number of points quickly becomes expensive to process. The second reason, and the more important one, is quality. The vast majority of point pairs join two different spheres, or involve a clutter point, and are simply wrong. Each such edge is a spurious piece of evidence that the solver then has to recognise and overrule. A complete graph does not just cost more to solve; it is mostly noise.

Gating avoids both problems. Instead of connecting all pairs, we admit an edge only between two points that are close enough to *plausibly* belong to the same object: nearby within a frame, or a short step apart between consecutive frames. Distant pairs, which are almost certainly unrelated, get no edge at all. This keeps the number of edges roughly linear in M instead of quadratic, and it means the graph handed to the solver is already largely free of wrong edges. The only question is how to set the “close enough” threshold, which is the subject of the next step. Figure 4 shows the effect on a real instance.



Figure 4: The gated instance graph in the interactive application: after gating, edges are drawn only between points close enough to plausibly share an object, leaving a sparse graph instead of the dense all-pairs one.

2.2.2 Setting the gate from the data

The gate needs a distance threshold: pairs closer than it get an edge, pairs farther do not. We could set this threshold by hand from the simulation settings, since we know the noise level, but a tracker running on real data would not have that luxury. So we estimate the threshold from the observed cloud itself, which also makes the method adapt automatically when the data changes.

The estimate is built from *nearest-neighbour distances*: for a given point, the distance to its single closest neighbour. We collect two separate samples of these, because gating happens in two situations. For edges *within* a frame, the sample $\mathcal{D}_s$ holds, for every point, the distance to its nearest neighbour in

the same frame. For edges *between* consecutive frames, the sample $\mathcal{D}_m$ holds, for every point in frame t , the distance to its nearest neighbour in frame $t+1$. Both are computed with a k -d tree, a spatial index that finds nearest neighbours in $O(M \log M)$ time instead of the $O(M^2)$ of checking all pairs, built with periodic boundaries so that the distances respect the torus wrapping of Equation (1).

The key observation is that each of these samples has two peaks (it is *bimodal*). Points that belong to a sphere sit inside a dense cluster, so their nearest neighbour is very close: these produce a peak of small distances, the *object mode*. Clutter points are scattered and isolated, so their nearest neighbour is far: these produce a spread of large distances, the *clutter mode*. A good threshold lives in the valley between the two peaks, large enough to sit above the object mode, so real within-object edges are kept, but small enough to stay below the clutter mode, so clutter-to-anything edges are rejected. Figure 5 shows the two modes and this valley on a real instance.

To place the threshold in that valley automatically, we use a robust estimator. Writing $\mathcal{D}$ for either sample, the threshold is

$$\rho = \underbrace{\text{med}(\mathcal{D})}_{\text{centre of object mode}} + c \cdot \underbrace{\text{MAD}(\mathcal{D})}_{\text{spread of object mode}}, \quad \text{MAD}(\mathcal{D}) = \text{med}_i(|d_i - \text{med}(\mathcal{D})|). \quad (6)$$

Here $\text{med}(\mathcal{D})$ is the median of the distance sample and $\text{MAD}(\mathcal{D})$ is its median absolute deviation, the median of how far each distance sits from the median, which is a robust measure of spread. The dimensionless margin $c > 0$ sets how many spreads above the centre the threshold is placed. In words: start at the typical within-object distance and step outward by c measures of its spread.

The reason for the median and MAD, rather than the ordinary mean and standard deviation, is robustness to the clutter. The median and MAD are order statistics: they depend on the *middle* of the sorted sample, not on the extreme values. As long as object points outnumber the clutter that reaches into the object mode, the large clutter distances cannot drag the estimate upward, however large they happen to be. The mean and standard deviation, by contrast, are sums over every value, so the clutter tail inflates them; a threshold built that way would widen exactly as clutter density rises, which is the opposite of what we want. This robustness holds up to a 50% breakdown point: it survives until clutter becomes the majority of the sample.

A useful property falls out of Equation (6) for free: it is *scale-equivariant*. If the whole cloud were rescaled by some factor λ , every distance would scale by λ , so med , MAD and therefore ρ would all scale by λ too. The gate tracks the natural scale of the data without any manual adjustment. There is a second consequence. The between-frame gate needs no explicit velocity term, even though it is really asking “how far can an object move in one frame?”. Each distance in $\mathcal{D}_m$ already *is* a one-step displacement plus observation noise, so the typical motion is measured directly from the data rather than assumed from a model.

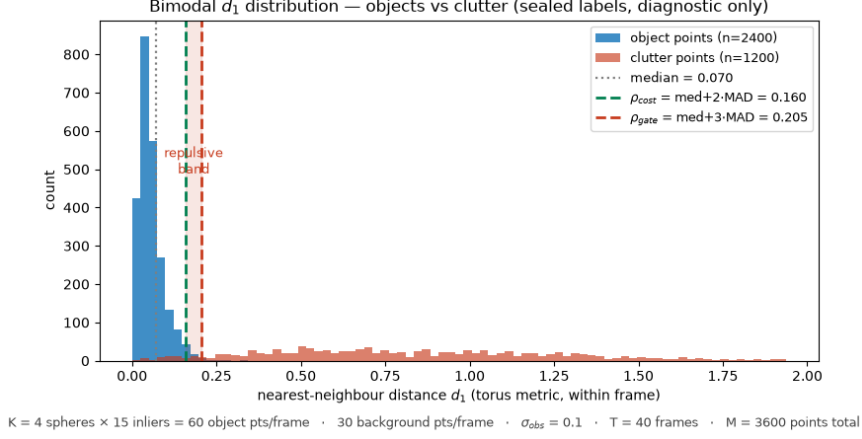


Figure 5: Within-frame nearest-neighbour distances d_1 for a representative instance, split by the sealed labels (shown only for illustration; the estimator sees the combined, unlabelled sample). The object mode (small distances) and the clutter mode (large distances) are clearly separated, and the estimated radii $\rho_{0,\text{in}}$ and ρ_{in} fall in the valley between them.

The same threshold serves a second purpose as a lightweight clutter filter. A point with fewer than three neighbours inside the within-frame gate, in its own frame, cannot be part of any real cluster, so we drop it from the graph before building edges. Rather than deleting such points, we keep them as isolated singletons, which keeps the predicted labels aligned with the ground truth for scoring. Object points have on the order of n_{in} neighbours in their frame and easily survive; isolated clutter does not, so this step removes background points that would otherwise act as bridges between distinct tracks.

2.2.3 From one threshold to two: gating versus cost

So far the gate does one job: decide which pairs get an edge. But an edge that exists still needs a *cost*, a number saying how strongly its two endpoints should be pulled into the same cluster, or pushed apart. A natural first attempt is to reuse the gate threshold for both jobs: admit an edge if the points are within ρ , and make the cost positive (attractive) below ρ , negative (repulsive) above it. The problem is that this attempt never produces a single negative cost. By construction, every admitted edge is already closer than ρ , so every admitted edge would be attractive. The graph would contain only “pull together” evidence and no “push apart” evidence at all.

That failure is not harmless. With only attractive edges, the solver has no reason to ever keep two clusters separate: it would merge everything that is transitively connected, which reduces the whole method to finding connected components of the gated graph. A single chain of clutter points bridging two spheres would then be enough to fuse them into one cluster. To avoid this we need genuine repulsive evidence, which means the cost must be able to turn negative for edges that are still inside the gate.

The fix is to use *two* thresholds from the same estimator instead of one. Both come from Equation (6), differing only in the margin c :

- the **gate radius** ρ , computed with a generous margin $c_{\text{gate}} = 3$, decides which pairs become edges at all;
- the **cost radius** ρ_0 , computed with a smaller margin $c_{\text{cost}} = 2$, sets the distance at which an edge’s cost switches from attractive to repulsive.

Because $c_{\text{cost}} < c_{\text{gate}}$, the cost radius sits inside the gate radius ($\rho_0 < \rho$). The cost of an admitted edge with length d_e is then

$$c_e = \alpha(\rho_0 - d_e), \quad (7)$$

where $\alpha > 0$ is a fixed scale factor. Reading off the sign: edges shorter than ρ_0 have $c_e > 0$ and are attractive; edges between ρ_0 and ρ have $c_e < 0$ and are repulsive; pairs beyond ρ are never admitted. This matches the multicut objective of Equation (5): minimising $\sum_e c_e y_e$ leaves an attractive edge joined ($y_e = 0$) and cuts a repulsive one ($y_e = 1$), so a positive cost pulls its endpoints together and a negative one pushes them apart. The gap between the two radii is exactly the band of repulsive edges the solver was missing (Figure 6). Applying this to the two samples gives a gate radius and cost radius for within-frame edges, ρ_{in} and $\rho_{0,\text{in}}$, and likewise for between-frame edges, ρ_{mot} and $\rho_{0,\text{mot}}$.

The value of α turns out not to matter, and we set it to 1. The reason is that the solver’s decisions depend only on the signs of summed costs and on their order, not on their absolute size (Section 2.3); scaling every cost by the same α changes neither, so it leaves the final partition untouched.

The ordering of the two margins deserves a plain statement, because the two thresholds are guarding against opposite errors. The gate controls *admission*, and its dangerous error is dropping a true edge: an edge that is never created can never be recovered by the solver, so a true within-object link lost at the gate is lost for good. The gate is therefore set generously, favouring recall. The cost radius controls an admitted edge’s *sign*, and its error is milder: a single edge given the wrong sign can still be outvoted by the many correct edges around it, because the solver acts on sums. The cost radius is therefore set more tightly, favouring precision. The generous threshold protects against the irreversible mistake; the tight one makes the reversible decision. That asymmetry is the whole reason there are two.

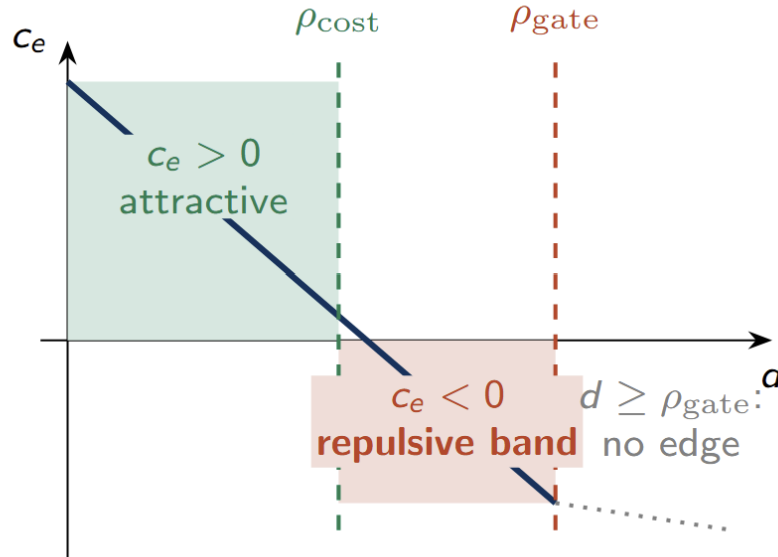


Figure 6: Edge cost $c_e = \alpha(\rho_0 - d_e)$ versus distance. Edges with $d_e < \rho_0$ are attractive; admitted edges in the band $\rho_0 < d_e < \rho$ are repulsive; pairs with $d_e \geq \rho$ are not admitted. Keeping ρ_0 below ρ is what makes the instance a genuine multicut rather than a connected-components problem.

The result is one spacetime graph $G = (V, E)$ whose vertices are the filtered detections and whose edges are the gated within-frame and between-frame pairs, each with a signed cost.

2.2.4 Calibrating the cost margin

The gate margin $c_{\text{gate}} = 3$ follows from the recall argument above: the gate should be generous. The cost margin c_{cost} sets the decision boundary and is calibrated. We swept $c_{\text{cost}} \in \{0.5, 1.0, 1.5, 2.0, 2.25, 2.5\}$ with everything else fixed, over six conditions (three clutter levels crossed with two noise levels), three seeds each, scoring end-to-end VI on inliers, with the selection rule fixed in advance: minimise the worst case across conditions. The result is a clear valley. A small margin puts the cost zero-crossing inside the

object mode, so true edges vote against merging and objects fragment; a large margin pushes it into the clutter scale, so over-merging returns. The minimum sits at $c_{\text{cost}} = 2.0$, interior to the grid, with all six conditions agreeing there. We use that value throughout. Figure 7 shows the six condition curves and the worst-case envelope that the selection rule minimises.

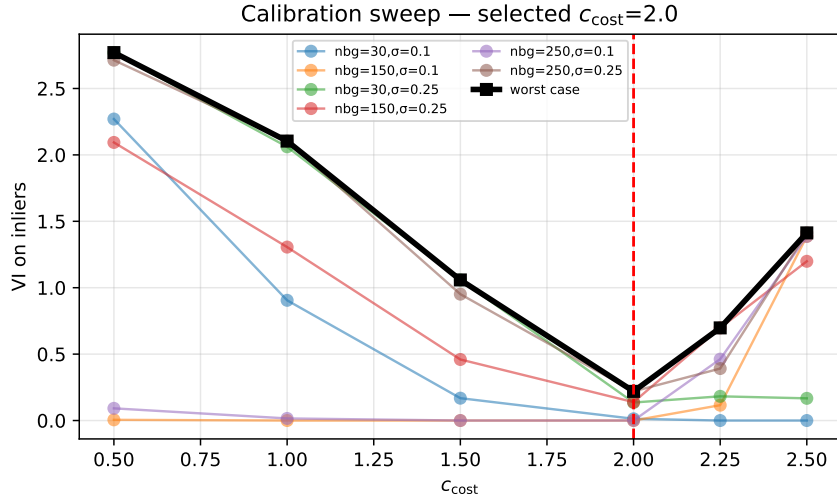


Figure 7: Calibration of the cost margin c_{cost} . VI on inliers against c_{cost} across the six conditions (three clutter levels $\times$ two noise levels), with the worst-case curve highlighted. The minimum of the worst case is interior to the grid at $c_{\text{cost}} = 2.0$, the value used throughout.

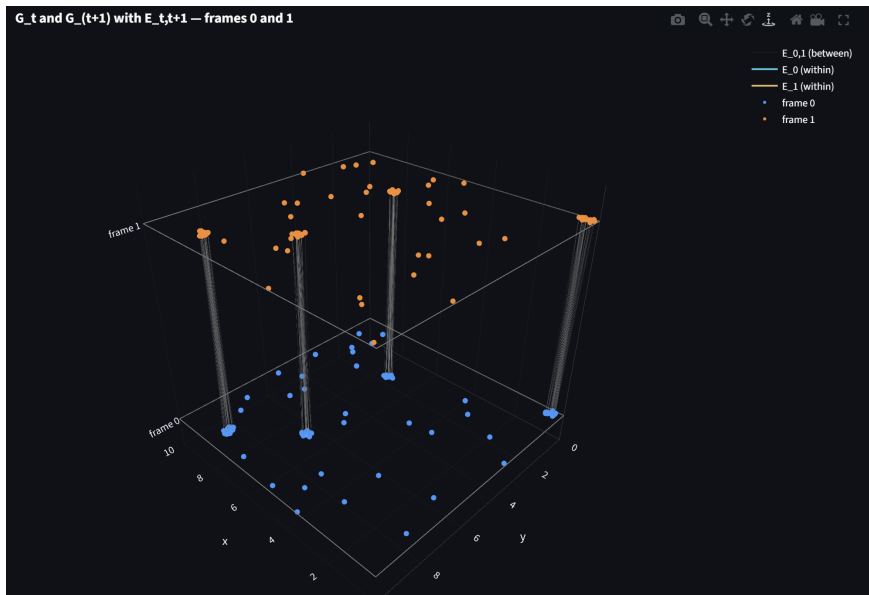


Figure 8: The gated spacetime graph passed to the solver: vertices are the filtered detections, and edges are the gated within-frame and between-frame candidate pairs, each carrying a signed cost. This is the instance that greedy additive edge contraction partitions into tracks.

2.3 Solving the instance

The instance produced by Section 2.2 is a graph $G = (V, E)$ whose vertices are the filtered detections and whose edges carry signed costs. We partition it with greedy additive edge contraction (GAEC) [1]. The algorithm recovers the tracks directly: each cluster of the final partition is one track. Figure 8 shows one such instance over a consecutive frame pair.

Every detection begins as its own cluster. For each pair of adjacent clusters a and b , the solver maintains their *join value*,

$$\text{join}(a, b) = \sum_{e \in E(a, b)} c_e, \quad (8)$$

the summed cost of all edges running between them. Because our costs are signed, this sum mixes the two kinds of evidence from the instance builder: attractive within- and between-frame edges push the join value up, repulsive edges pull it down. A positive join value means the accumulated evidence favours merging the two clusters into one track; a negative value favours keeping them apart.

At each step GAEC contracts the pair with the largest join value, merging them into a single cluster, and updates the join values of the merged cluster's neighbours by summing the contributions each still has to the two former clusters. It repeats until no pair has a positive join value, at which point no merge would lower the multicut cost of Equation (5) any further. The remaining clusters are returned as the tracks (Algorithm 1).

Algorithm 1: Greedy additive edge contraction (GAEC).

```

Input: graph  $G = (V, E)$ , edge costs  $c : E \rightarrow \mathbb{R}$ 
Output: labelling  $\lambda : V \rightarrow \mathbb{N}$ 
foreach  $v \in V$  do
    |  $\lambda(v) \leftarrow v$ ;                                // each detection its own cluster
end
foreach  $\{i, j\} \in E$  do
    |  $\text{join}(i, j) += c_{ij}$ ;                            // sum edge costs, + and -
end
 $Q \leftarrow \{(a, b) : \text{join}(a, b) > 0\}$ ;                // max-priority queue
while  $Q \neq \emptyset$  do
    |  $(a, b) \leftarrow$  pair in  $Q$  of maximum join value;
    | if  $\text{join}(a, b) \leq 0$  then
    | | break;                                        // no beneficial merge left
    | end
    | contract  $b$  into  $a$ ; let  $l$  be the absorbed cluster;
    | foreach neighbour  $n$  of  $l$  do
    | |  $\text{join}(a, n) += \text{join}(l, n)$ ;                    // fold sums into survivor
    | | if  $\text{join}(a, n) > 0$  then
    | | | insert  $(a, n)$  into  $Q$ 
    | | end
    | end
end
return  $\lambda$ 

```

Summing over all edges between two clusters, rather than acting on individual edges, is what lets the within-frame and between-frame costs resolve against each other without any hand-set weight: a pair joined by several weak attractive edges and one strong repulsive edge is decided on the balance of the two. This is also where the two-radius cost of Section 2.2.3 does its work. The repulsive edges it creates are what eventually drive some join value negative and halt the contraction; were every edge attractive, no join value could turn negative, and GAEC would merge every connected cluster, collapsing the partition to the connected components of the gated graph. The signed costs and the greedy contraction are therefore two halves of one design, in which the instance builder supplies the repulsive evidence and the solver acts on it.

Using Union-Find for cluster membership and a priority queue over join values, the contraction runs in $O(|E| \log |E|)$. On the instances of Section 3 a solve takes 0.6–1.1 s, decreasing as noise grows: heavier noise widens the estimated gates, removes more points at the core-point filter, and shrinks the edge set, so the graph GAEC processes gets smaller as the problem gets harder. Figure 9 shows a solved partition in the interactive application, with each recovered cluster drawn in its own colour.

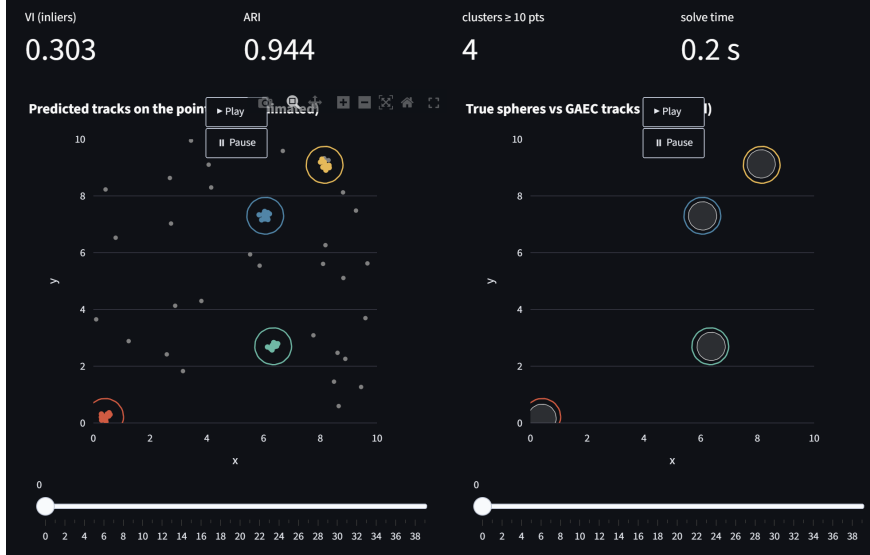


Figure 9: The solved partition in the interactive application: the clusters recovered by GAEC, coloured by track. Each colour is one cluster of the final partition, i.e. one recovered object.

2.4 Constraint handling

The exact multicut of Equation (5) is subject to three families of constraints, and our approach handles each differently.

The first is the **cycle constraints** of Equation (5) themselves, which enforce that “belonging to the same track” is transitive: within any cycle, a single edge cannot be the only one cut. These are satisfied automatically by our solver. GAEC returns its result as a labelling that assigns each detection to a cluster, and any such labelling is already a valid partition; the cut assignment read from it therefore cannot violate a cycle constraint. No explicit enforcement or repair step is required.

The remaining two families are the **no-split** and **no-join** constraints, which forbid a track from forking into two across a frame, or two tracks from fusing into one. We relax these: they are simply not imposed. Doing so reduces the constrained program to the plain minimum-cost multicut that GAEC solves, and follows the same trade-off as Tang et al. [4], giving up temporal consistency in return for a formulation an efficient solver can handle. The consequence is that a returned track could, in principle, fork or fuse across a frame, since nothing in the solver forbids it.

In our pipeline the gate makes this unlikely. A fork or fusion requires edges between points of different spheres, and excluding exactly those pairs is what gating does; on the default configuration only 0.9% of the positive edges join points from different spheres. The relaxation therefore holds in practice up to $\sigma_{\text{obs}} = 0.20$, beyond which it begins to break down, as quantified in Section 3. We report these violations where they occur rather than repairing them after the fact, and we do not measure the gap between our greedy solution and the exact optimum.

Having built the instance and solved it, we now turn to how its accuracy is measured.

2.5 Evaluation

Evaluating the tracker requires three things: a metric that scores a predicted partition against the ground truth, a set of parameters we vary to probe the method’s behaviour, and a procedure that ties the two together into repeatable experiments. We take them in that order.

2.5.1 Metric

The solver returns a partition of the observed points into clusters with arbitrary integer ids; the ground truth is a different partition of the same points, labelled by sphere. Evaluation must therefore compare two

partitions of one set *without* assuming any correspondence between their labels, since the solver’s cluster “3” need not be the true sphere “3”. The metric is chosen for exactly that property.

We use the **variation of information** (VI). For a true partition $\mathcal{T}$ and a predicted partition $\mathcal{P}$,

$$\text{VI}(\mathcal{T}, \mathcal{P}) = H(\mathcal{T} \mid \mathcal{P}) + H(\mathcal{P} \mid \mathcal{T}), \quad (9)$$

where $H(\cdot \mid \cdot)$ is a conditional entropy. VI is a true metric on the space of partitions: it is zero if and only if the two partitions are identical, it is symmetric, and it satisfies the triangle inequality [2]. It also needs no matching step between cluster ids, so the solver’s free choice of labels costs nothing. Values are in nats (the natural-log convention).

The reason VI suits this project is that its two terms name the two ways the tracker can fail. $H(\mathcal{T} \mid \mathcal{P})$ is the **merge error**: the uncertainty left about the true sphere once the predicted cluster is known, which is large exactly when the tracker has fused two spheres into one cluster. $H(\mathcal{P} \mid \mathcal{T})$ is the **split error**: the uncertainty about the predicted cluster given the true sphere, which is large when the tracker has fragmented one sphere across several clusters. A single VI value can therefore be read not just for the *size* of the error but for its *kind*, and we report the two terms separately throughout.

One consequence anchors the results. Fusing all K equally sized spheres into a single cluster gives $\text{VI} = \ln K$; with $K = 4$ this is $\ln 4 \approx 1.386$ nats. A measured VI above this ceiling is only possible if the partition also carries split error, so the ceiling is a direct test of whether a breakdown is pure fusion or fusion together with fragmentation.

VI has no fixed upper bound, since it grows with the number of clusters, which makes a raw value hard to place on an absolute scale. We therefore report the **adjusted Rand index** (ARI) alongside it as a bounded, chance-corrected check: ARI is 1 for a perfect match and near 0 for a random one. VI stays the primary metric because its split/merge decomposition carries the failure analysis; ARI calibrates it for the reader.

Finally, both metrics are computed **over inlier points only**. Background clutter has no true sphere identity, carrying the label -1 by construction, so scoring it would measure how the metric treats unlabelled points rather than how well the tracker recovers tracks. Clutter still participates fully in instance construction; it is excluded only from the final score.

2.5.2 What we vary

The synthetic generator exposes the quantities that control how hard the tracking problem is, and these are the axes along which we stress-test the method. Each is varied independently in the experiments of Section 3.

- **Number of spheres K** : the number of objects in the scene. More spheres means more object boundaries, and therefore more places where the gate might admit an edge between two different spheres.
- **Observation noise σ_{obs}** : the spread of the inlier detections around each true centre. Larger σ_{obs} inflates the object clusters, and once they are wide enough to overlap the background, the within-object and object-to-clutter distance scales become hard to separate.
- **Background clutter**: the number of uniform background points per frame. This is the density of the distractor population, and it turns out to affect not only the difficulty but the gate estimator itself (Section 3).
- **Motion regime**: selected by the motion-noise scale σ_{mot} . With $\sigma_{\text{mot}} = 0$ the spheres move ballistically, in straight lines between collisions; with $\sigma_{\text{mot}} > 0$ their heading drifts each frame in an inertial random walk. The two regimes test whether the method depends on predictable motion.
- **Object speed**: how far a sphere travels between frames. The motion gate assumes this is small relative to the clutter spacing; varying speed tests where that assumption holds.

- **Inlier density:** the number of detections n_{in} each sphere produces per frame, i.e. how well populated each true cluster is.
- **Pre-filter threshold min_neighbors :** the one solver-side parameter we vary, namely the number of in-frame neighbours a point must have within ρ_{in} to survive the noise pre-filter.

The remaining parameters are fixed: the two gate multipliers c_{gate} and c_{cost} , the cost slope α , and the world size L . The multipliers are set as described in Sections 2.2.3 and 2.2.4; $\alpha = 1$ is a lossless normalisation; and L is fixed because the gate estimator is scale-equivariant, so its value does not affect accuracy at constant density.

2.5.3 Procedure

Every experiment follows the same one-factor-at-a-time design: a single parameter from Section 2.5.2 is varied across a range while all others are held at their defaults, so that any change in the metric is attributable to that one parameter. This isolation is what lets each experiment answer a single question, how the method responds to that one axis, rather than confounding several effects at once.

Each individual run is a full pass of the pipeline: the generator synthesises a cloud for the given parameters, the instance is built with gate radii estimated from that cloud, the greedy solver partitions it, and the result is scored against the sealed ground truth by (9). Every run is seeded, so the whole procedure is reproducible bit for bit. To separate genuine trends from the noise of a single random draw, each configuration is run on three seeds and reported as the mean with its standard deviation; a trend is only treated as real if it is larger than that seed-to-seed spread.

The default configuration, from which each experiment departs along one axis, is $K = 4$ spheres, $n_{\text{in}} = 20$ inliers and 50 background points per frame, over $T = 60$ frames, at a reference noise chosen per experiment. The experiments themselves, their ranges, and the resulting tables and figures are presented in Section 3.

2.6 The interactive application

The pipeline is also exposed as a Streamlit application (`app.py`), which runs the same `tbcc` package that produced every number in this report, so there is no separate implementation. Sliders control the world, the observation model, and the gate multipliers; the panels run from the hidden ground-truth world through the observed cloud, the instance graph, the solved partition, and per-run diagnostics. Stages are cached separately, so moving a gate multiplier re-solves without re-simulating, which makes the interface responsive enough to explore. Both gate multipliers are exposed as sliders and the estimated radii are shown for the current cloud, so a reader can confirm that no threshold is a tuned constant. The application defaults to a lighter world than the grid ($T = 40$, 15 inliers, 30 clutter, 3,600 points) and builds with `cyclic=False`, so its on-screen numbers should not be read against the results table. Figure 10 shows the full interface.

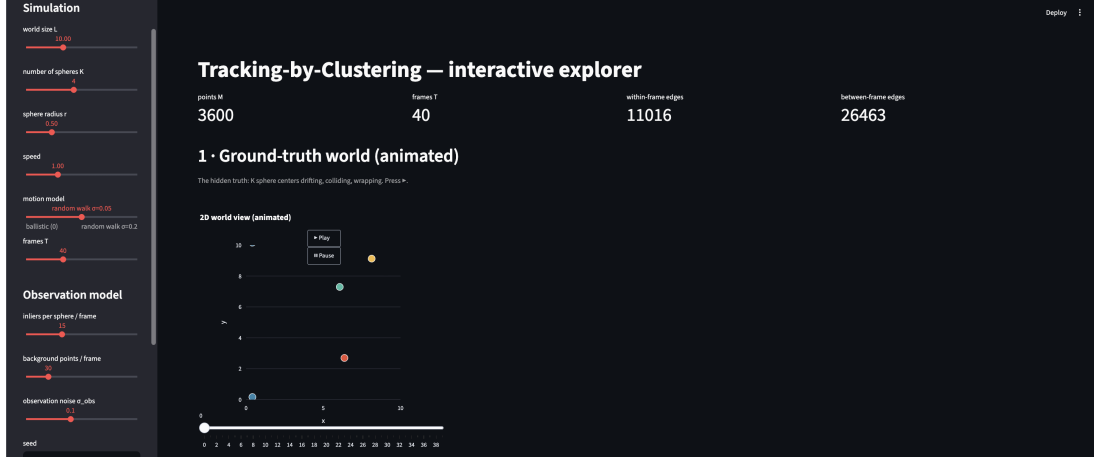


Figure 10: The interactive application. Sliders on the left control the world, the observation model, and the gate multipliers; the main panels show the ground-truth world, the observed cloud, the instance graph, and the solved partition, each recomputed as the sliders move. []

3 Results

We characterise the method by the sensitivity analysis of Section 2.5: each experiment varies one generative parameter while holding the rest at their defaults, and reports the variation of information (VI) on inliers, averaged over three seeds, together with its split/merge decomposition where informative. We first present the main sweep over observation noise and motion regime, then vary the number of objects, the clutter density, the object speed, the inlier density, and the pre-filter threshold in turn, and close with a closer analysis of how the method fails.

3.1 Observation noise and motion regime

The main experiment crosses the two motion regimes with four noise levels. Table 1 reports VI with its split/merge decomposition, the adjusted Rand index, and the connected-components baseline VI_{CC} — the partition the solver would return if the repulsive edges were removed, included to isolate what the two-radius cost contributes. Figure 11 plots the same VI values against noise, with the total-fusion reference line.

Table 1: Main sweep: two motion regimes $\times$ four noise levels, three seeds each (24 runs). VI on inliers, in nats; split and merge sum to VI up to rounding. VI_{CC} is the connected-components baseline.

model	σ_{obs}	VI (mean $\pm$ std)	VI_{CC}	ARI	split	merge
ballistic	0.05	0.091 ± 0.017	0.084	0.985	0.091	0.000
ballistic	0.10	0.185 ± 0.029	0.100	0.969	0.185	0.000
ballistic	0.20	0.400 ± 0.025	0.663	0.930	0.398	0.002
ballistic	0.40	1.386 ± 0.166	1.242	0.406	0.781	0.605
random walk	0.05	0.086 ± 0.015	0.080	0.986	0.086	0.000
random walk	0.10	0.175 ± 0.027	0.095	0.971	0.175	0.000
random walk	0.20	0.382 ± 0.021	0.656	0.934	0.379	0.003
random walk	0.40	1.507 ± 0.029	1.428	0.301	0.760	0.746

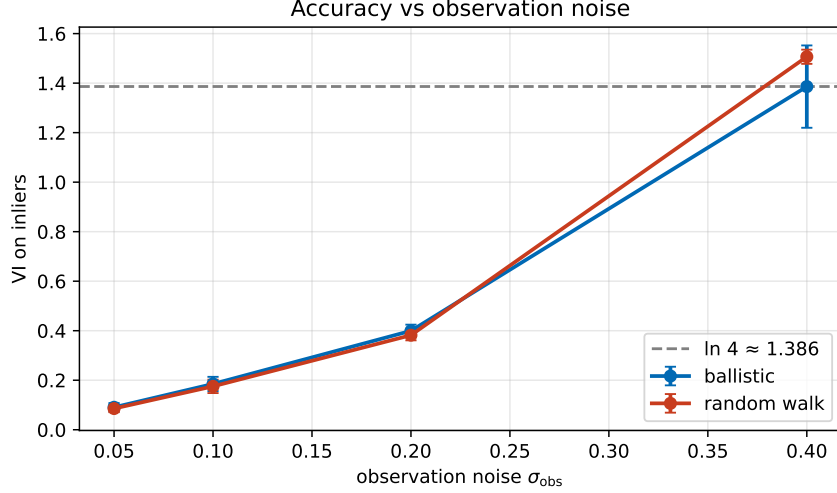


Figure 11: VI against observation noise for both motion regimes, mean over three seeds with ± 1 standard deviation, and the total-fusion reference line at $\ln 4 \approx 1.386$.

The sweep separates into two regimes. At $\sigma_{\text{obs}} \leq 0.10$ the tracker recovers all four spheres in every run: VI stays near 0.09 at $\sigma_{\text{obs}} = 0.05$ and 0.18 at $\sigma_{\text{obs}} = 0.10$, with ARI above 0.96 and merge error exactly 0.000 throughout — no two spheres are ever fused. The small residual is pure split error from the core-point filter, analysed in Section 3.7. Between $\sigma_{\text{obs}} = 0.20$ and 0.40 lies a breakdown window: VI climbs from about 0.4 to roughly 1.4–1.5 and ARI falls from above 0.93 to near 0.30–0.40. The transition is sharp — one doubling of the noise carries the tracker from near-exact recovery to the total-fusion ceiling of $\ln 4 \approx 1.386$. The two motion regimes behave almost identically at every noise level, because nothing is tuned for either: the gates are estimated per dataset, so the radii move with the motion. The connected-components baseline VI_{CC} tells a more interesting story, taken up in the Discussion (Section 4.2): it is marginally lower than VI at low noise but markedly higher at $\sigma_{\text{obs}} = 0.20$, where distinct spheres first begin to bridge.

3.2 Number of objects

Varying the number of spheres K from 2 to 12, at the default noise, tests how the method scales with scene complexity. Figure 12 reports accuracy and runtime across that range.

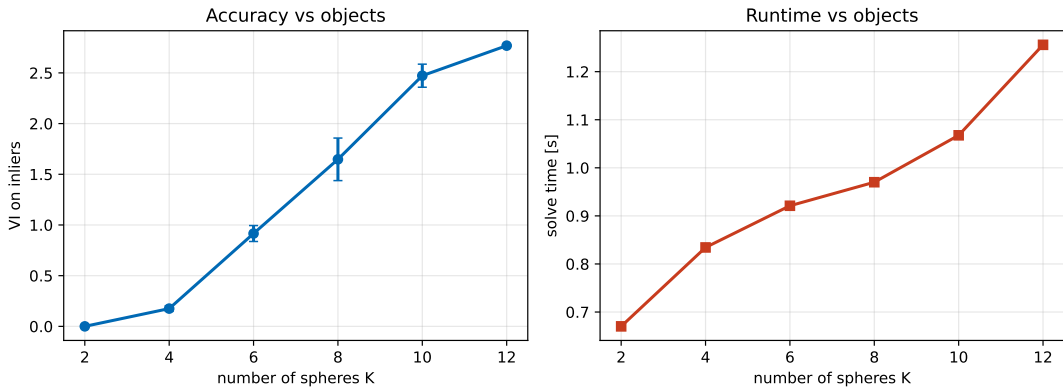


Figure 12: Accuracy (left) and solve time (right) as the number of spheres grows, mean over three seeds.

Accuracy degrades gracefully as the scene fills. With $K = 2$ the recovery is exact (VI 0.000); at $K = 4$ it is the familiar 0.18; but VI then rises steadily — roughly 0.9 at $K = 6$, 1.6 at $K = 8$, and 2.5–2.8 by

$K = 10\text{--}12$, where the partition fragments into dozens of clusters rather than the target handful. The mechanism is geometric: more spheres pack more object boundaries into the same world, and each boundary is another place where the gate can admit a cross-object edge, so fusions and fragmentations multiply. Runtime, by contrast, grows only mildly — from 0.7 s at $K = 2$ to 1.3 s at $K = 12$ — since the edge count scales with the number of points, not with the difficulty of separating them. The method tracks reliably up to about $K = 4$ at this noise and world size; beyond that, accuracy falls off well before runtime becomes a concern.

3.3 Background clutter

The clutter density is varied from none to well above the default. The estimator depends on clutter to locate the two distance scales (Section 2.2.2), so its effect is not monotone. Table 2 reports the sweep and Figure 13 plots it.

Table 2: Clutter sweep at fixed noise ($\sigma_{\text{obs}} = 0.10$, random-walk motion, three seeds). Accuracy is non-monotone: too little clutter starves the gate estimator, too much lets clutter dominate.

background / frame	VI (mean $\pm$ std)	split	merge	clusters ≥ 10
0	4.701 ± 0.037	4.701	0.000	70
10	3.555 ± 0.051	3.555	0.000	46
25	1.572 ± 0.130	1.572	0.000	11
50	0.175 ± 0.027	0.175	0.000	4
100	0.000 ± 0.000	0.000	0.000	4
200	0.000 ± 0.000	0.000	0.000	224

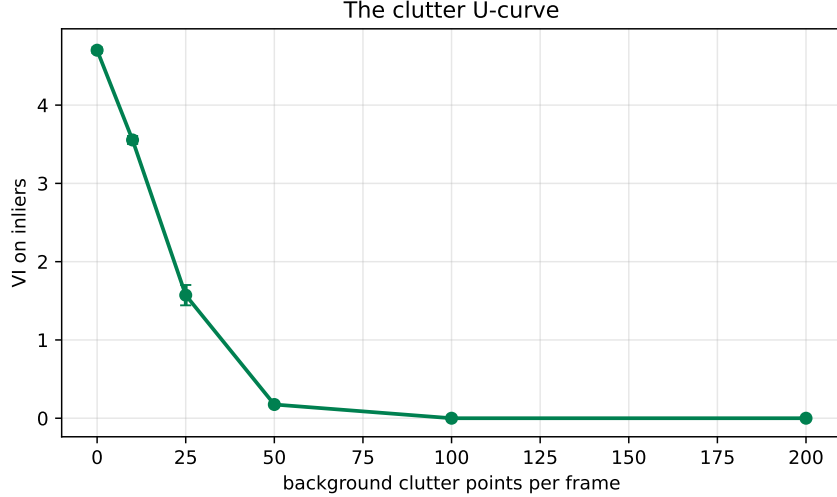


Figure 13: VI as a function of background clutter density.

The result is the U-shape the estimator predicts, and its left arm is striking: with *no* clutter the tracker fails completely (VI 4.70, fragmenting into some seventy clusters), and adding background *improves* accuracy, monotonically, until VI reaches zero around 100 points per frame. The reason is the gate estimator. With little or no clutter, the nearest-neighbour distances contain only within-object distances, so $\text{med} + c \cdot \text{MAD}$ produces a tiny radius that severs true within-object edges and shatters every sphere; the merge error stays exactly zero throughout, confirming the failure is pure fragmentation. The estimator needs the clutter present to see the second distance scale it calibrates against. The right arm appears past the default: at 200 points per frame VI is still nominally zero on inliers, but the clutter itself now forms hundreds of spurious clusters, the first sign of the median being pulled toward the clutter scale. The safe

operating band is therefore an interval, not a floor — enough clutter to define the two scales, but not so much that it dominates the sample.

3.4 Object speed

Object speed is varied at the default noise to probe the assumption behind the between-frame gate — that a sphere moves less than the clutter spacing between frames. Figure 14 reports the result.

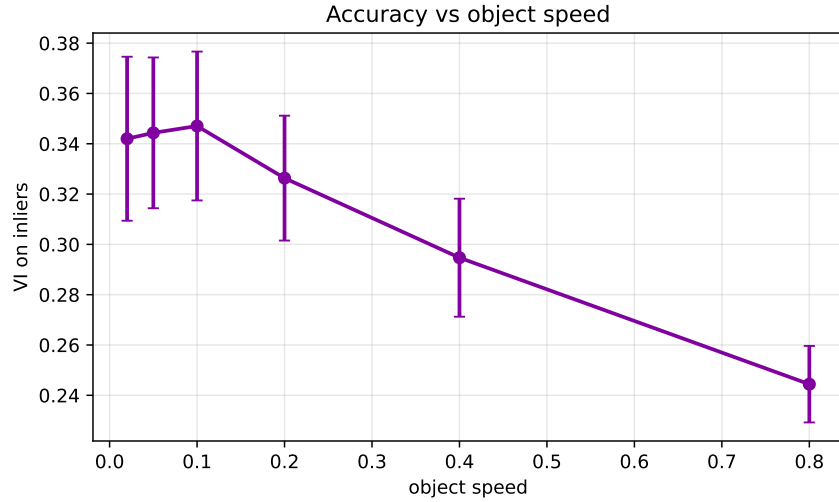


Figure 14: VI as a function of object speed, mean over three seeds.

This experiment produced a result we did not expect: VI does not rise with speed over the tested range — if anything it drifts slightly *down*, from about 0.34 at low speed to 0.24 at the highest. The between-frame gate absorbs the motion rather than breaking on it, and the reason follows from how the gate is built. Because its radius is estimated from the observed one-step displacements (Section 2.2.2) rather than assumed from a fixed motion model, a faster object simply produces a larger typical displacement, and the estimated radius grows with it. The gate adapts to the speed instead of being outrun by it. We report this as a genuine finding: the assumption we set out to stress — that displacement stays below the clutter spacing — is not the binding constraint in this range, because the data-driven gate rescales itself to whatever displacement it observes.

3.5 Inlier density

The number of detections per sphere per frame, n_{in} , is varied at the default noise, testing how sparse an object can be before it fails to track. Figure 15 reports the sweep.

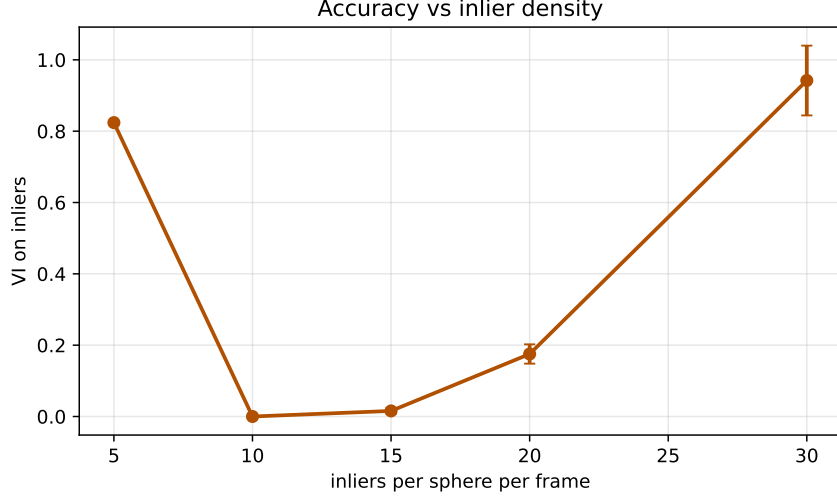


Figure 15: VI as a function of the number of inliers per sphere per frame, mean over three seeds.

The dependence on inlier density is non-monotone, with a sweet spot around 10–15 detections per sphere. Below it, sparse objects fail: at $n_{\text{in}} = 5$, VI rises to 0.82 and the scene fragments into some forty clusters, because thin objects fall below the pre-filter’s three-neighbour threshold and are stranded as singletons. Above it, accuracy also degrades somewhat — VI reaches 0.94 at $n_{\text{in}} = 30$ — which reflects the interaction of denser clusters with the fixed gate radius rather than a failure to detect the objects (all four remain present). The practical reading is that the method needs on the order of ten detections per object per frame for reliable tracking, and that both extremes, too sparse and too dense, are handled less well than the middle of the range the default sits in.

3.6 Pre-filter threshold

The one solver-side parameter, `min_neighbors`, is varied at two noise levels to justify the default of three. Table 3 reports both noise levels.

Table 3: Pre-filter sweep at two noise levels (random-walk motion, three seeds). Too low admits clutter; too high drops real inliers and raises split error. The default of three balances the two.

<code>min_neighbors</code>	VI at $\sigma_{\text{obs}} = 0.10$	VI at $\sigma_{\text{obs}} = 0.20$
1	0.120	0.314
2	0.138	0.340
3	0.175	0.382
5	0.313	0.571
8	0.749	1.169

The sweep confirms the trade-off behind the default. A low threshold admits almost everything: at `min_neighbors` = 1 VI is lowest (0.12 at $\sigma_{\text{obs}} = 0.10$), but the filter is then barely removing clutter. As the threshold rises, VI grows steadily — 0.18 at three, 0.31 at five, and 0.75 at eight — because a stricter filter increasingly strands genuine inlier points as singletons, raising the split error; the same pattern holds, shifted upward, at $\sigma_{\text{obs}} = 0.20$. The lowest VI is at the loosest setting, but the value of the filter is precisely to remove the clutter that a loose setting keeps — clutter that would otherwise bridge tracks in the harder regimes. Three is the value that filters effectively while stranding few real points, and it is used throughout.

3.7 Failure anatomy

A VI curve says the tracker degrades; the split/merge decomposition says how. Reading the main sweep (Table 1) together with the sensitivity experiments, the failure has a clear anatomy.

At low noise ($\sigma_{\text{obs}} \leq 0.10$) the merge error is exactly zero in every run: no predicted cluster ever blends two spheres. All the residual VI is split error, and its source is the core-point filter — requiring three neighbours within the gate strands a few genuine inlier points as singletons, which count as fragments of their sphere. This is the pre-filter working as designed, and it is the reason there is any error at all at $\sigma_{\text{obs}} = 0.05$; the pre-filter sweep (Table 3) makes the same point from the other side, as raising the threshold trades clutter removal for exactly this kind of split error.

At $\sigma_{\text{obs}} = 0.20$ the objects remain intact — five to six clusters, merge error at or below 0.003 — with the growing VI coming almost entirely from boundary points shaved off by noise. This is the last noise level at which the structure holds, and it is where the repulsive edges earn their place: the connected-components baseline jumps to 0.66 here (Table 1), because gated components have begun to bridge distinct spheres, and only the repulsive edges keep GAEC from following those bridges into fusion.

At $\sigma_{\text{obs}} = 0.40$ the breakdown is mixed, and the $\ln 4$ ceiling makes this precise. The measured VI — about 1.39 for ballistic, 1.51 for random-walk — sits at or *above* the total-fusion value of $\ln 4 \approx 1.386$. Total fusion of all four spheres into one cluster would give exactly $\ln 4$ and no more, so a value at or above it is only possible because split and merge error are now both large and comparable (each around 0.6–0.75 nats). The diffuse clouds are simultaneously fragmenting and fusing, and the ceiling is what proves the breakdown is both at once rather than fusion alone.

Across all of these, the single deepest cause is the gate estimator’s reliance on a visible gap between the object and clutter distance scales — the same dependence that the clutter ablation (Table 2) exposes from the opposite direction. When noise inflates the object scale until it meets the clutter scale, or when clutter is removed until there is no second scale to find, the estimator loses the valley it reads, and the method fails in the manner the decomposition makes visible.

4 Discussion

4.1 What the results show

The system recovers all four spheres, without being told how many, through $\sigma_{\text{obs}} = 0.10$ with merge error exactly zero in every run, and continues to do so at $\sigma_{\text{obs}} = 0.20$ with the objects still intact. This is near-exact recovery across a $4\times$ range of observation noise and across both motion regimes, from a tracker that sees only positions and timestamps and is told nothing about the number of objects or their dynamics. That the number of tracks is recovered rather than set is a direct consequence of the signed-cost formulation: contraction stops when the data offers no positive evidence, so $K = 4$ is an output.

The sensitivity experiments locate the edges of this operating region. The method holds to about four objects at the tested world size and degrades as the scene fills; it needs on the order of ten detections per object per frame; and, more surprisingly, it is essentially indifferent to object speed, because the data-driven between-frame gate rescales itself to whatever displacement it observes rather than assuming a fixed one. The most counter-intuitive result is the clutter ablation: removing the background does not help the tracker but destroys it, because the gate estimator needs the clutter present to locate the second distance scale it calibrates against. Taken together, these define an envelope, a region of object count, density, and noise within which the method recovers tracks reliably, rather than a single headline number.

4.2 Why the two radii matter

The VI_{CC} column of Table 1 measures what the repulsive edges buy, and the answer is more interesting than a uniform improvement. At $\sigma_{\text{obs}} \leq 0.10$ the connected components of the positive-cost subgraph score marginally *better* than GAEC (0.08–0.10 against 0.09–0.19): every gated component is already a single sphere, so the repulsive edges have nothing to prevent, and their only effect is to shave a few

boundary points. At $\sigma_{\text{obs}} = 0.20$ the picture inverts, with GAEC scoring 0.400 and 0.382 against the components' 0.663 and 0.656, because this is the first level at which gated components begin to bridge distinct spheres, and connected components has no mechanism to stop. At $\sigma_{\text{obs}} = 0.40$ both partitions have failed and the comparison is uninformative. The repulsive edges are best understood as insurance: they cost a little precision where nothing is difficult, and they supply the only barrier to fusion in the regime the method has to survive.

4.3 Limitations

Four limits should be stated. First, the **no-split and no-join constraints are relaxed**: we solve the plain multicut, report any violations rather than repairing them, and do not measure the gap to the exact optimum. Second, the method **scales to only a handful of objects** at the tested world size, with accuracy falling off steadily beyond about four spheres, as more object boundaries give the gate more opportunities to admit a cross-object edge. Third, the **gate estimator degrades when the two distance scales converge**. By $\sigma_{\text{obs}} = 0.40$ the within-object and object-to-clutter distance populations overlap, the valley the estimator reads has essentially ceased to exist, and no distance-only criterion can separate them; this is a limit of the feature, not the solver. Fourth, the estimator **assumes objects are locally dense clusters embedded in clutter**. The clutter ablation shows it depends on clutter being present to locate the two scales, and the inlier-density sweep shows objects must be well enough populated to survive the pre-filter, so a clutter-free or single-detection-per-object setting would need the estimator rethought.

4.4 Outlook

Several directions follow. Learned edge costs would replace the assumed logistic cost with one fitted to the data, for which the greedy already provides the partitioning step. Explicit no-split and no-join handling would close the relaxation, at the cost of a harder problem. A comparison against the exact integer program would bound the greedy's optimality gap on small instances. And velocity coherence in the between-frame cost, scoring whether a displacement is consistent with a cluster's motion rather than only its length, is the natural way to push past the breakdown window, where distance alone stops separating the objects.

References

- [1] M. Keuper, E. Levinkov, N. Bonneel, G. Lavoué, T. Brox, and B. Andres. Efficient decomposition of image and mesh graphs by lifted multicut. In *IEEE International Conference on Computer Vision (ICCV)*, pp. 1751–1759, 2015.
- [2] M. Meilă. Comparing clusterings by the variation of information. In *Learning Theory and Kernel Machines (COLT)*, pp. 173–187. Springer, 2003.
- [3] S. Tang, B. Andres, M. Andriluka, and B. Schiele. Subgraph decomposition for multi-target tracking. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5033–5041, 2015.
- [4] S. Tang, B. Andres, M. Andriluka, and B. Schiele. Multi-person tracking by multicut and deep matching. In *European Conference on Computer Vision (ECCV) Workshops*, pp. 100–111. Springer, 2016.